

WEEK 4 MAKE-UP CLASS · REGRESSION ANALYSIS

Linear Regression Models

Least squares · gradient descent · evaluation · regularization · polynomial features

Class edition · the KOSPI Colab notebook walked through cell by cell (Python · scikit-learn · Colab)

Prof. Keehong Rhee

Today in Three Messages

What you should be able to say at the end of this session

1

Fit — minimize the squared error

Closed form (normal equation) or iterative (gradient descent): same answer, different cost.

2

Judge — out of sample, and honestly

RMSE, MAE and R^2 on data the model never saw.

3

Control — match complexity to the data

Ridge, Lasso, ElasticNet and polynomial features trade a little bias for a lot less variance.

Easy to fit, easy to fool: the evaluation design decides what you learn.

Roadmap

Six sections — theory first, then the same ideas on real market data

#	Section	Key question
01	Linear regression basics	What line fits best, and how do we find it?
02	Model evaluation	How wrong is the model — and compared to what?
03	Basic lab	The scikit-learn four-step pattern
04	Regularization	How do we stop a model from memorizing noise?
05	Polynomial regression	Can a linear model capture curves?
06	KOSPI case lab	What does the full workflow look like on real data?

01

SECTION 01

Linear Regression Basics

Model · cost function · normal equation · gradient descent

Vocabulary First

Every later slide uses these words — the right column shows where each lives in scikit-learn

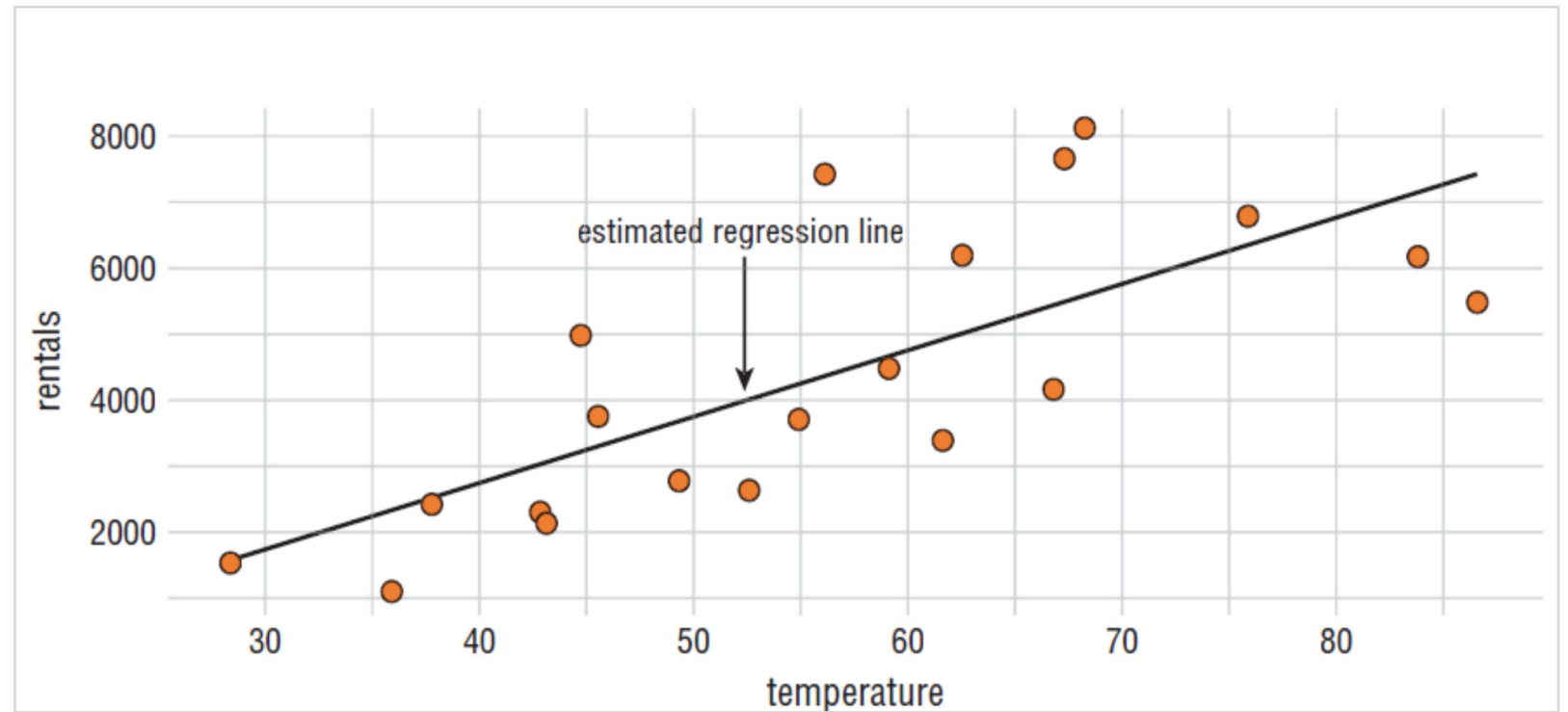
Term	Meaning	In code
Target y (dependent variable)	What we want to predict	<code>y, y_train</code>
Feature x (independent variable)	Input used to predict y	<code>X, X_train</code>
Weight / coefficient w	Change in $\hat{y}$ for +1 unit of x , other features fixed	<code>model.coef_</code>
Intercept / bias b	Value of $\hat{y}$ when all features are 0	<code>model.intercept_</code>
Prediction $\hat{y}$	Model output for a given x	<code>model.predict(X)</code>
Residual $e = y - \hat{y}$	Observed minus fitted (computable)	<code>y_test - y_pred</code>
Error ε	Unobservable noise in the true process	—

Simple Linear Regression

One feature x , one target y : find the straight line (regression line) that best describes their relationship

What you see

- y : daily bike rentals
- x : temperature
- Dot = one day
- Line = fitted $\hat{y} = wx + b$

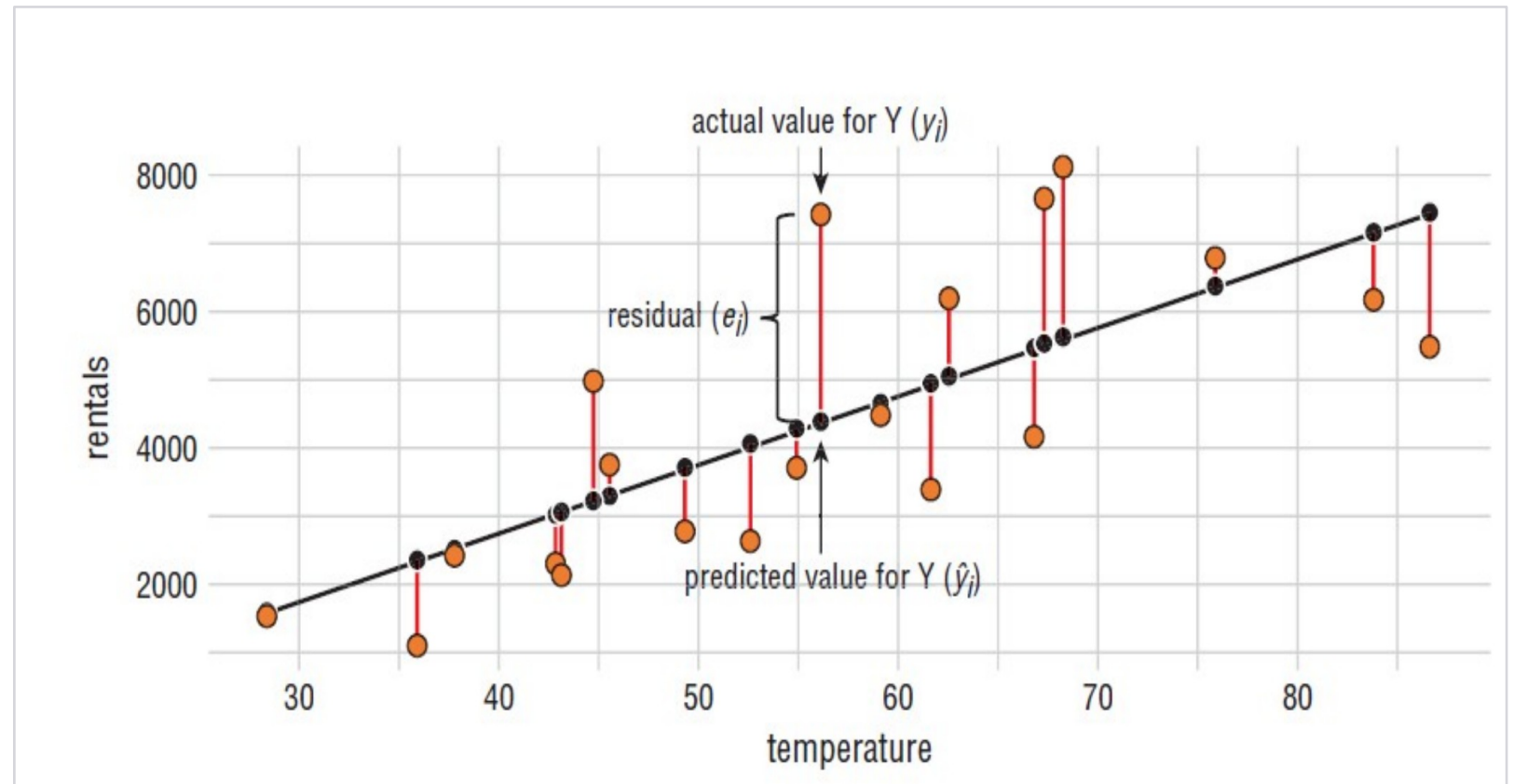


Residuals and Least Squares

Residual = actual – predicted. OLS picks the line with the smallest sum of squared residuals

Read the chart

- Orange dot: actual y
- Black dot: predicted $\hat{y}$
- Red bar: residual e
- Square, then sum



The Linear Regression Equation

From one feature to n features — and a compact vector form

Simple

$$\hat{y} = w x + b$$

Multiple

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

Vector form

$$\hat{y} = \sum_{j=0}^n w_j x_j = \mathbf{w}'\mathbf{x} \quad (x_0 = 1, w_0 = b)$$

Read it as

- w: slope — effect of each x on $\hat{y}$
- b: intercept — $\hat{y}$ when every $x = 0$

Fixed from the original slide

- The intercept was written twice ($b_0 \cdot x_0 \dots + b_0$). Use '+ b' or ' $x_0 = 1$ ', not both.

The Cost Function: Mean Squared Error

Also called loss, objective or error function — the single number the algorithm drives down

$$J(w, b) = \underbrace{\frac{1}{2}}_{\text{makes the derivative clean}} \underbrace{\frac{1}{m}}_{\text{average over } m \text{ training samples}} \sum_{i=1}^m \underbrace{(y_i - \hat{y}_i)^2}_{\text{squared error of sample } i}$$

Why squared errors?

- Misses of +10 and -10 cannot cancel
- Smooth → easy to differentiate
- Big misses cost much more (outlier-sensitive)

Note on 1/(2m)

- Strict MSE uses 1/m
- The extra ½ only cancels the 2 in the derivative
- Same minimizer w^* , b^* either way

Solution 1: The Normal Equation

Set the derivatives of J to zero and solve once, in matrix form

① Differentiate the cost

$$\frac{\partial J}{\partial w} = -\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i) x_i \qquad \frac{\partial J}{\partial b} = -\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)$$

② Set both to 0 and solve for all coefficients at once

$$\hat{\mathbf{w}} = \left(X'X \right)^{-1} X'y$$

Exact answer in one step — but the inversion gets expensive and unstable when features are many or highly correlated.

How it works

- X: $m \times (n+1)$ matrix with a column of 1s for b
- One linear solve — no learning rate, no loop
- sklearn LinearRegression solves this least-squares problem with a stable SVD solver

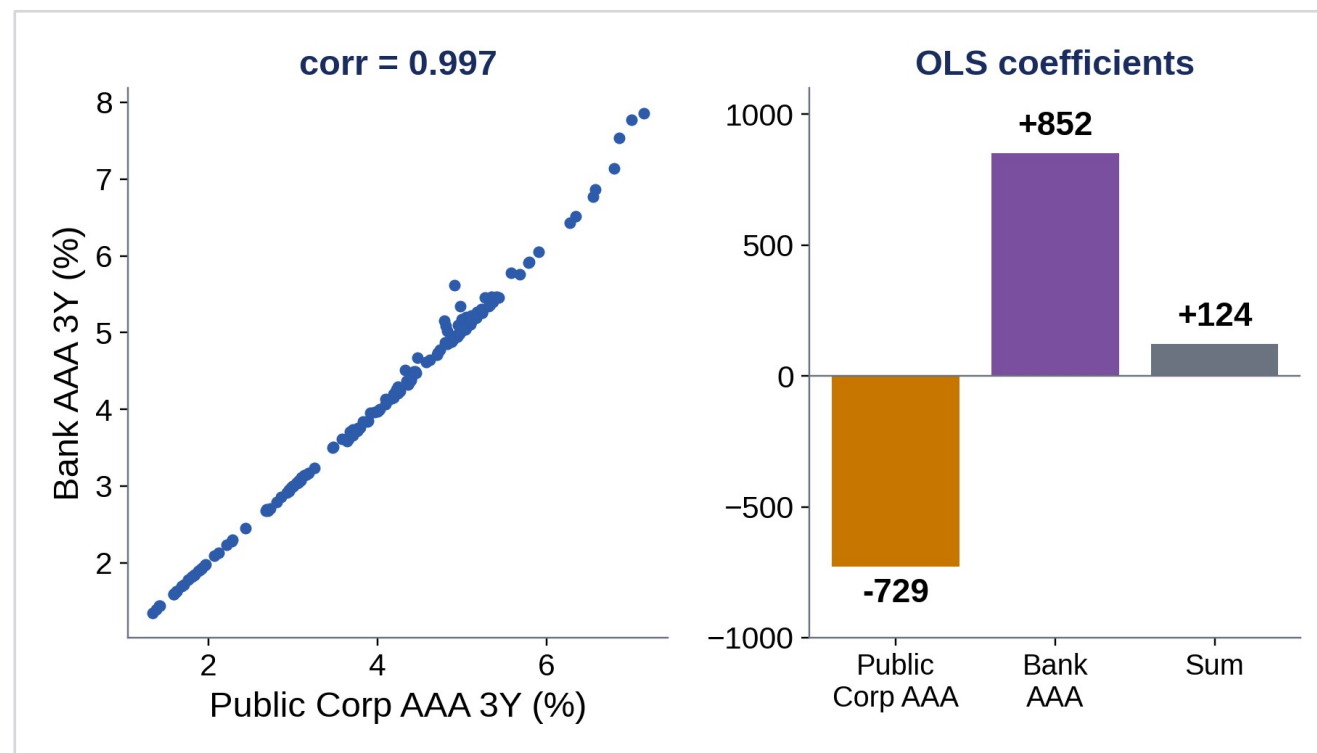
Pitfall: Multicollinearity

When features are highly correlated, $X'X$ is nearly singular and the coefficient estimates become unstable

$$\text{Var}(\hat{\mathbf{w}}) = \sigma^2 (X'X)^{-1}$$

What happens

- $\det(X'X) \rightarrow 0$, so its inverse — and the variance of $\hat{\mathbf{w}}$ — blow up
- Coefficients swing and flip sign; predictions can still look fine
- Fixes: drop or merge features, PCA, Ridge



KOSPI lab data: two AAA 3-year yields (corr 0.997) get weights -729 and +852 — their sum is only +124.

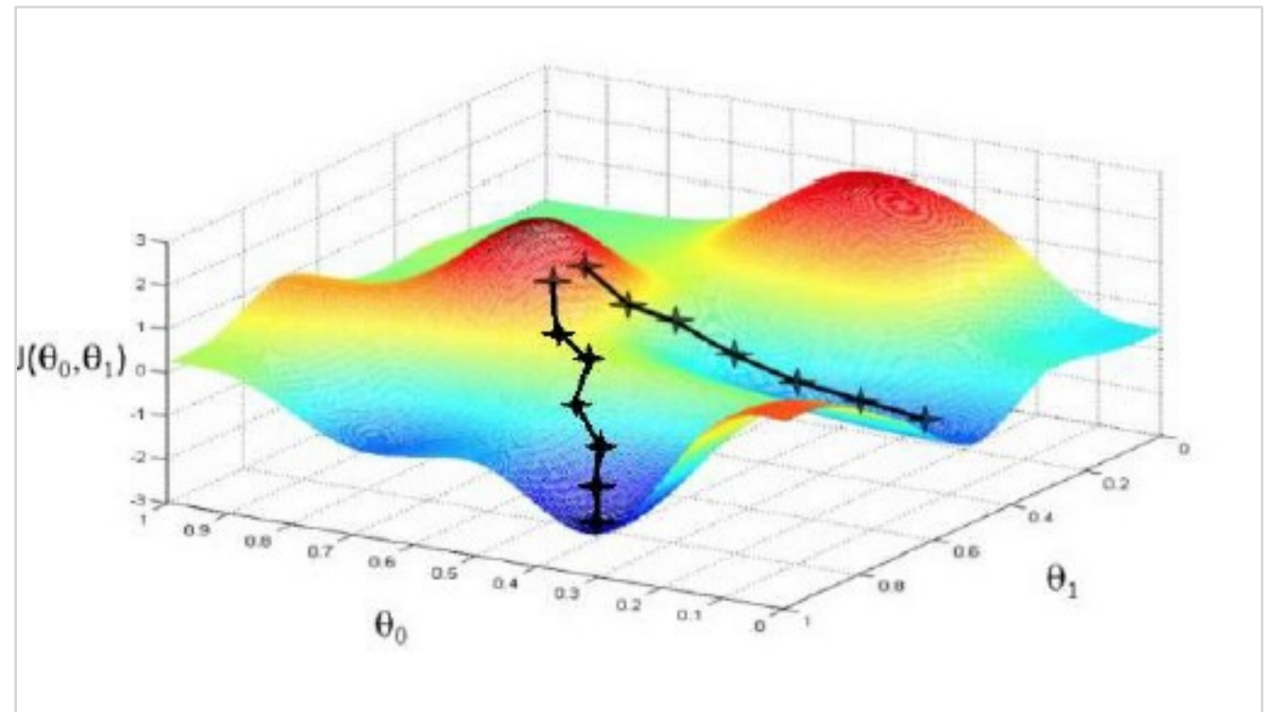
Solution 2: Gradient Descent

Start anywhere, step downhill along the negative gradient, repeat until the cost stops falling

$$w \leftarrow w - \eta \frac{\partial J}{\partial w} \quad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

The idea

- η (eta) = learning rate = step size
- Gradient points uphill, so move the opposite way
- Stop when J barely changes (convergence)
- The pictured surface has local minima; the OLS cost is a convex bowl with one global minimum

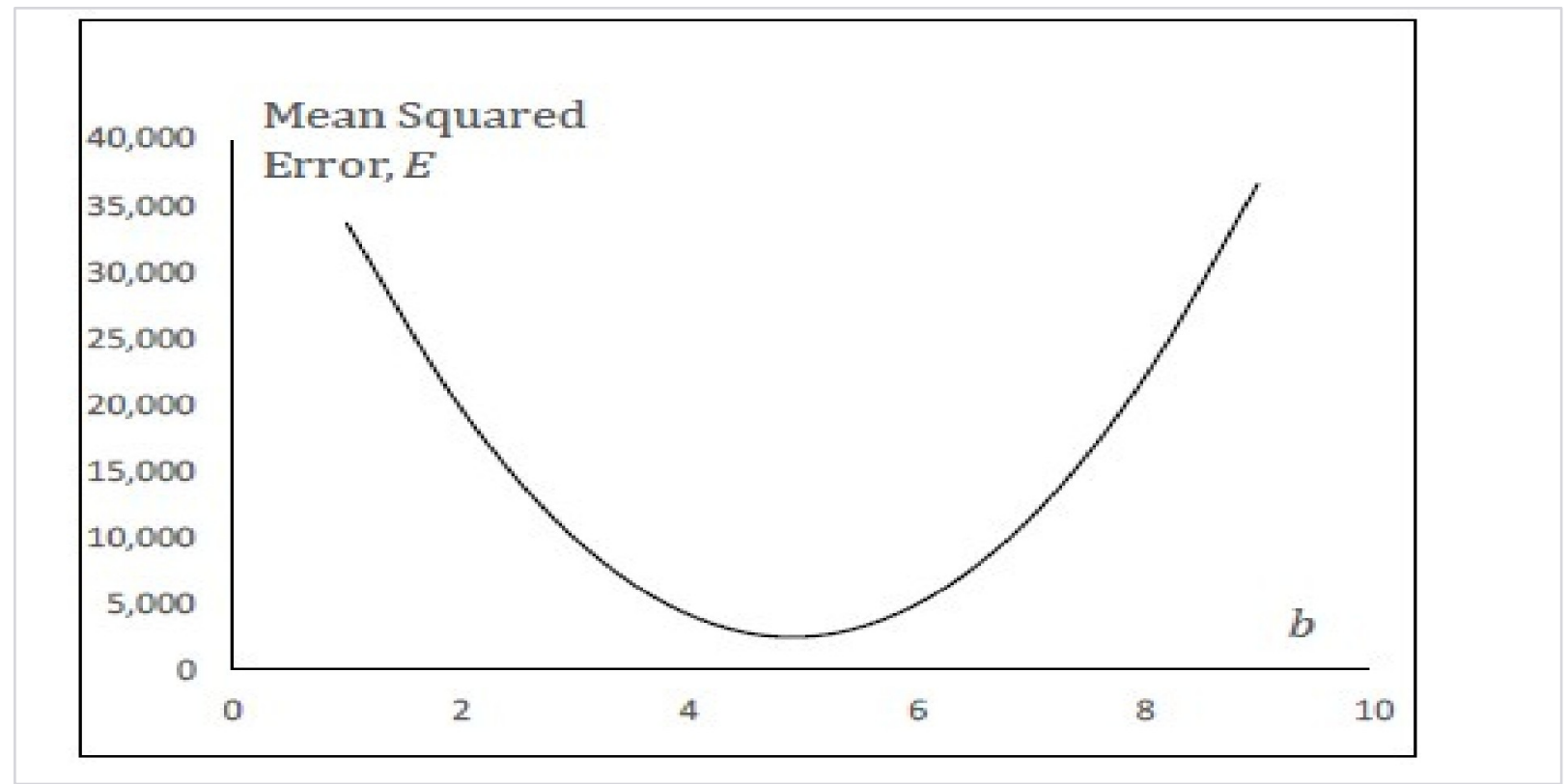


Gradient Descent Example: One Coefficient

Holding w fixed, MSE as a function of the intercept b is a parabola — GD walks down to its bottom

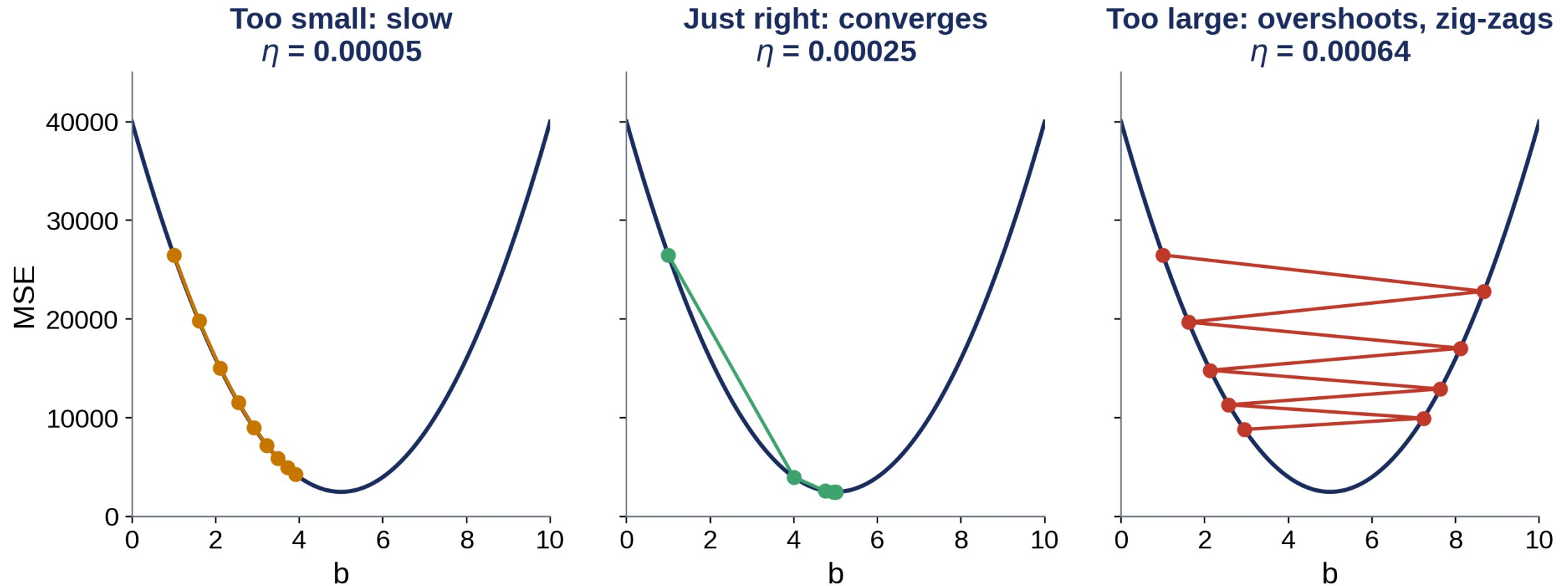
Read the chart

- x-axis: value of b
- y-axis: MSE
- Minimum near $b = 5$
- Slope sign gives the step direction



Choosing the Learning Rate η

Too small: painfully slow · just right: converges in a few steps · too large: overshoots, zig-zags, or diverges



Normal Equation vs. Gradient Descent

Same least-squares answer — choose by data size and flexibility

Aspect	Normal equation	Gradient descent
Solution	Exact, in one step	Iterative approximation
Settings to choose	None	Learning rate η , number of iterations
Feature scaling	Not required	Strongly recommended
Many features n	Solve cost $\sim n^3$ — slow for huge n	Cost per step $\sim m \cdot n$ — scales well
Huge or streaming m	Needs all data at once	Mini-batch / stochastic GD works
In scikit-learn	LinearRegression	SGDRegressor

02

SECTION 02

Model Evaluation

RMSE · MAE · MAPE · R^2 — and what each one hides

Before the Metrics: Where Do We Measure?

A metric is only as honest as the data it is computed on

Training set (in-sample)

- Used to fit w and b
- Error here is optimistic: the model has seen these rows

Test set (out-of-sample)

- Held out, never touched while fitting
- Error here estimates real-world performance

**Rule: fit everything — model and scaler — on the training set only.
The test set is used once, at the very end.**

Tuning settings such as α ? Carve a validation set out of the training data, or use cross-validation.

Prediction Error: RMSE, MAE, MAPE

RMSE is the most common forecast metric — but it is not the 'average error'; that is MAE

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| (\%)$$

RMSE

- Same units as y
- Squares first: big misses dominate
- Always $\text{RMSE} \geq \text{MAE}$

MAE

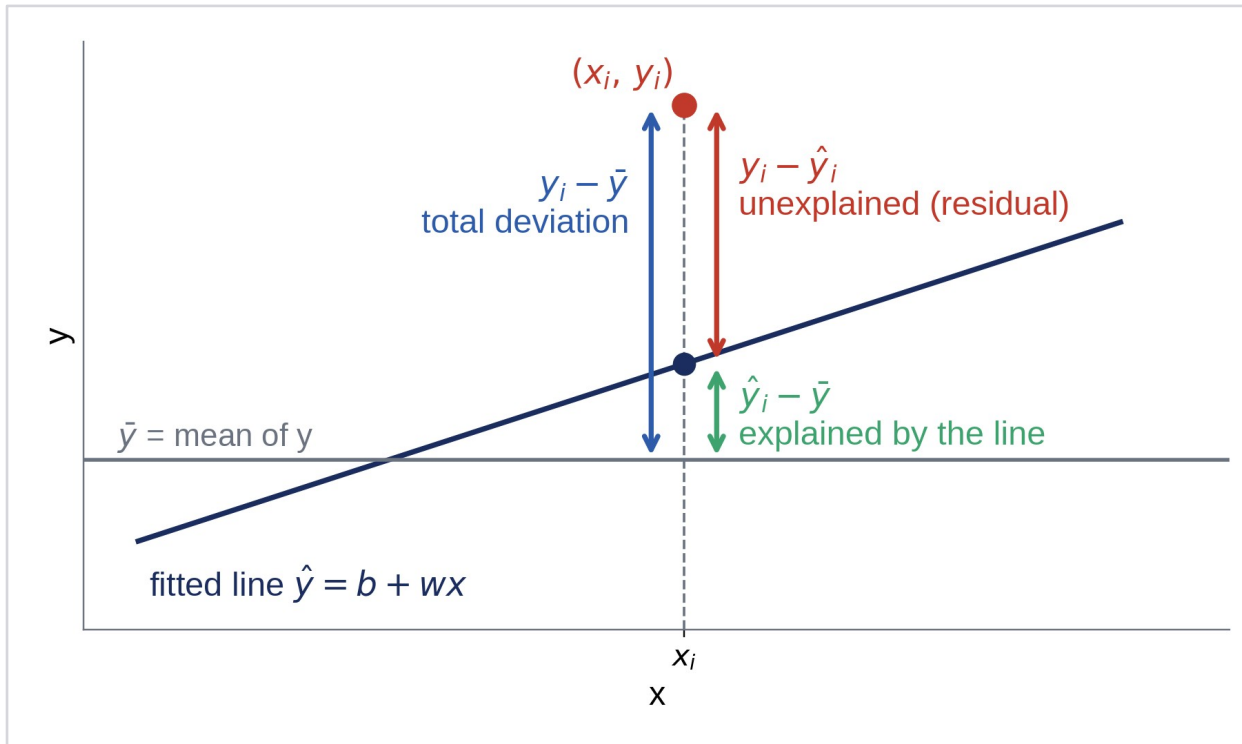
- Average absolute miss
- The 'typical error' reading belongs here
- Less sensitive to outliers

MAPE

- Scale-free, in %
- Undefined when $y = 0$
- Explodes when y is near 0

Explanatory Power: R^2

The share of the variation in y that the model explains



$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{SST (total)}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{SSR (explained)}} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{SSE (residual)}}$$

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = \frac{\text{SSR}}{\text{SST}}$$

Meaning

- $R^2 = 1$: explains all the variation
- $R^2 = 0$: no better than predicting the mean of y

R²: What the Formula Doesn't Tell You

Four caveats the original slide's ' $0 \leq R^2 \leq 1$ ' leaves out

$$R^2_{\text{test}} = 1 - \frac{\sum_{i \in \text{test}} (y_i - \hat{y}_i)^2}{\sum_{i \in \text{test}} (y_i - \bar{y}_{\text{test}})^2} \text{ can be } < 0$$

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

p = number of features, n = number of samples

Negative test R² is a verdict, not a bug.

Caveats

- $0 \leq R^2 \leq 1$ holds only in-sample, for OLS with an intercept
- On a test set R^2 can be negative: worse than predicting the test mean
- Adding a feature never lowers in-sample $R^2 \rightarrow$ use adjusted R^2 or a test set
- `r2_score(y_true, y_pred)` is not symmetric — argument order matters

03

SECTION 03

Basic Lab

The scikit-learn pattern: instantiate → fit → predict → evaluate

The scikit-learn Four-Step Pattern

Every estimator — linear, tree, neural net — is used with the same four calls

1 Instantiate

Choose the model and its settings

```
lr =  
LinearRegression()
```

2 Fit

Learn w and b from training data

```
lr.fit(X_train,  
       y_train)
```

3 Predict

Apply to unseen features

```
y_pred =  
lr.predict(X_test)
```

4 Evaluate

Compare with the truth

```
r2_score(y_test,  
         y_pred)
```

Dataset note

- The original lab used Boston Housing, which was removed from scikit-learn 1.2. This version uses the course KOSPI dataset (Section 06).

Lab 1: Simple Linear Regression

KOSPI explained by a single feature — the housing price index (random 80/20 split, seed 42)

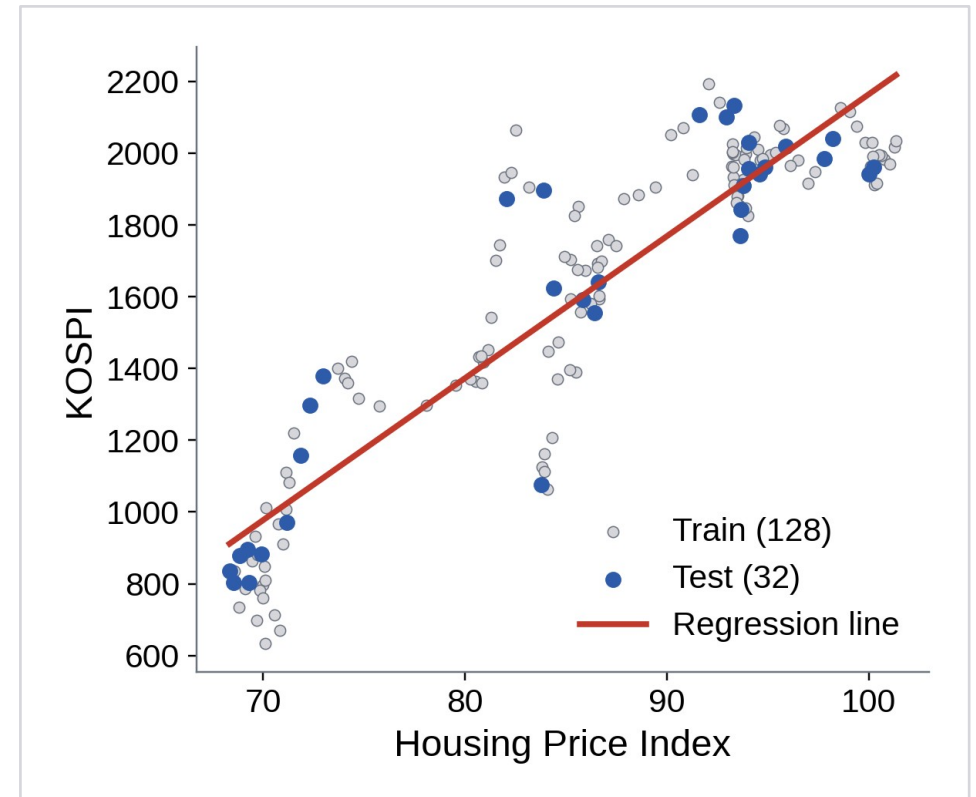
Python · Colab

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
f = ['Housing Price Index'] # [[ ]] keeps X 2-D

sim_lr = LinearRegression() # 1 instantiate
sim_lr.fit(X_train[f], y_train) # 2 fit
y_pred = sim_lr.predict(X_test[f]) # 3 predict
print('R2: {:.4f}'.format(r2_score(y_test, y_pred)))
print('w: {:.4f}, b: {:.4f}'.format(
    sim_lr.coef_[0], sim_lr.intercept_))
```

Output

```
R2: 0.8478
w: 39.5708, b: -1793.0459
```



+1 index point $\approx$ +40 KOSPI points

Lab 2: Multiple Linear Regression

Same split, all 29 features at once

Python · Colab

```
from sklearn.metrics import mean_squared_error
mul_lr = LinearRegression()
mul_lr.fit(X_train, y_train)          # 29 features
y_pred = mul_lr.predict(X_test)
print('R2: {:.4f}'.format(r2_score(y_test, y_pred)))
rmse = mean_squared_error(y_test, y_pred) ** 0.5
print('RMSE: {:.2f}'.format(rmse))
```

Output

```
R2: 0.9460
RMSE: 107.51
```

coef_ holds 29 weights; intercept_ = -2905.2

Model	Test R ²	RMSE
Simple (1 feature)	0.848	180.6
Multiple (29)	0.946	107.5

Before celebrating

- One random split is one draw — confirm with cross-validation
- Correlated rates get unstable weights

04

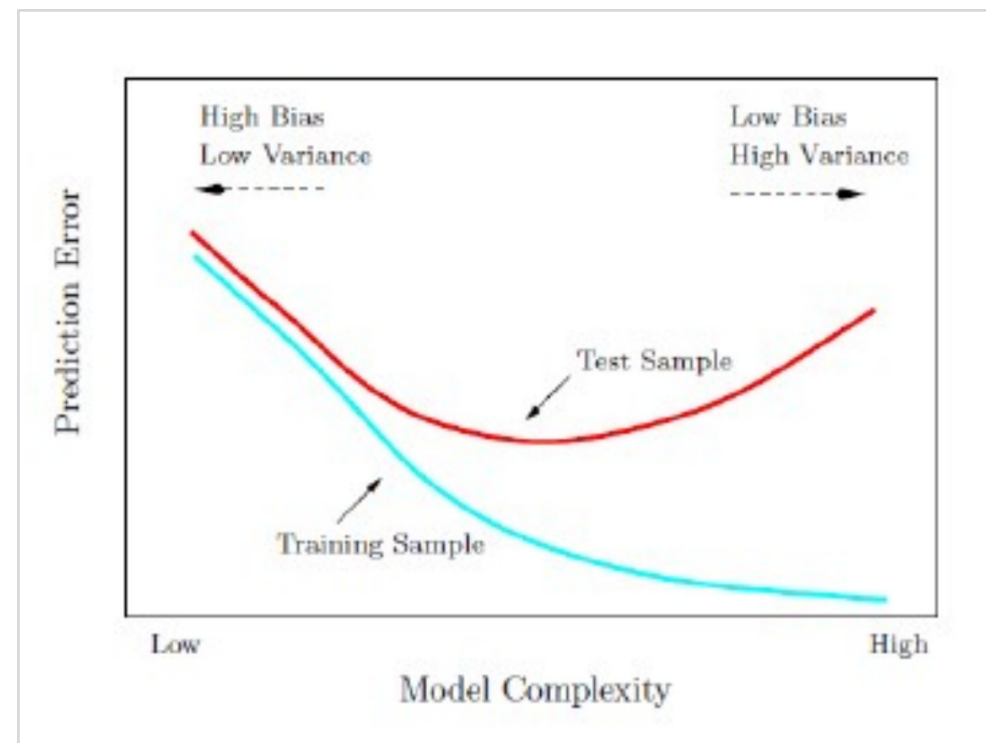
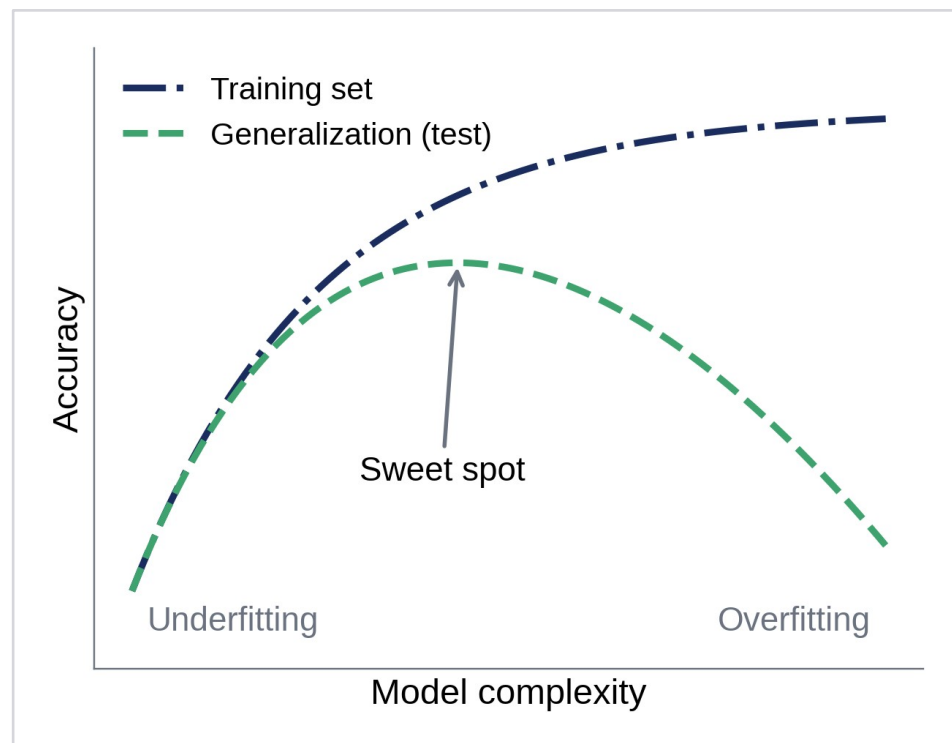
SECTION 04

Regularization

Ridge · Lasso · ElasticNet — putting a price on complexity

Why Regularize? Overfitting

With many features and few samples, a model memorizes noise: great on training data, poor on new data



Overfitting = poor generalization. Warning sign: many features p relative to samples n ($p > n$ is extreme).

Example: Who Buys a Boat?

Rule B = age > 45 AND (children < 3 OR not divorced) · data after Müller & Guido

Age	Cars	House	Children	Marital	Dog	Bought boat	Rule A: owns house (9/12)	Rule B (12/12)
66	1	yes	2	widowed	no	yes	✓ yes	✓ yes
52	2	yes	3	married	no	yes	✓ yes	✓ yes
22	0	no	0	married	yes	no	✓ no	✓ no
25	1	no	1	single	no	no	✓ no	✓ no
44	0	no	2	divorced	yes	no	✓ no	✓ no
39	1	yes	2	married	yes	no	✗ yes	✓ no
26	1	no	2	single	no	no	✓ no	✓ no
40	3	yes	1	married	yes	no	✗ yes	✓ no
53	2	yes	2	divorced	no	yes	✓ yes	✓ yes
64	2	yes	3	divorced	no	no	✗ yes	✓ no
58	2	yes	2	married	yes	yes	✓ yes	✓ yes
33	1	no	1	single	no	no	✓ no	✓ no

Model Complexity Must Match the Data

Rule B is 100% accurate on 12 rows — which is exactly why we should distrust it

With 12 customers

- Many rules fit perfectly (e.g., 'people aged 66, 52, 53 or 58 buy boats')
- Rule B's three conditions are tuned to these 12 people → likely overfit
- Simpler Rule A (75%) may generalize as well

With 10,000 customers

- If Rule B still held on 10,000 rows, we would trust it far more
- More — and more varied — data supports a more complex model
- More data usually beats cleverer tuning

Correction to the original slide

- It read 'using many variables can be more stable'. The lesson is the reverse: complexity is safe only when the data can support it.

Ridge Regression (L2)

Choose $\lambda \geq 0$ and minimize MSE plus a penalty on the squared size of the coefficients

$$J_{\text{ridge}}(\mathbf{w}) = \text{MSE}(\mathbf{w}) + \lambda \sum_{j=1}^n w_j^2$$

$$\hat{\mathbf{w}}_{\text{ridge}} = (X'X + \lambda I)^{-1} X'y$$

What happens as λ increases?

- $\lambda = 0$: ordinary least squares
- $\lambda \uparrow$: coefficients shrink toward 0 but never exactly 0
- $\lambda \rightarrow \infty$: all $w \rightarrow 0$, $\hat{y} \rightarrow \text{mean of } y$
- Bias $\uparrow$, variance $\downarrow$

Practical notes

- Standardize features first — the penalty is scale-sensitive
- The intercept b is not penalized
- Adding λI makes $X'X$ invertible $\rightarrow$ cures multicollinearity
- scikit-learn calls λ 'alpha'

Lasso Regression (L1)

Like Ridge, but the penalty is on the absolute size of the coefficients

$$J_{\text{lasso}}(\mathbf{w}) = \text{MSE}(\mathbf{w}) + \lambda \sum_{j=1}^n |w_j|$$

What the L1 penalty does

- Pushes some coefficients to exactly 0
- → removes less important features (built-in feature selection)
- Sparse model → easier to interpret

Practical notes

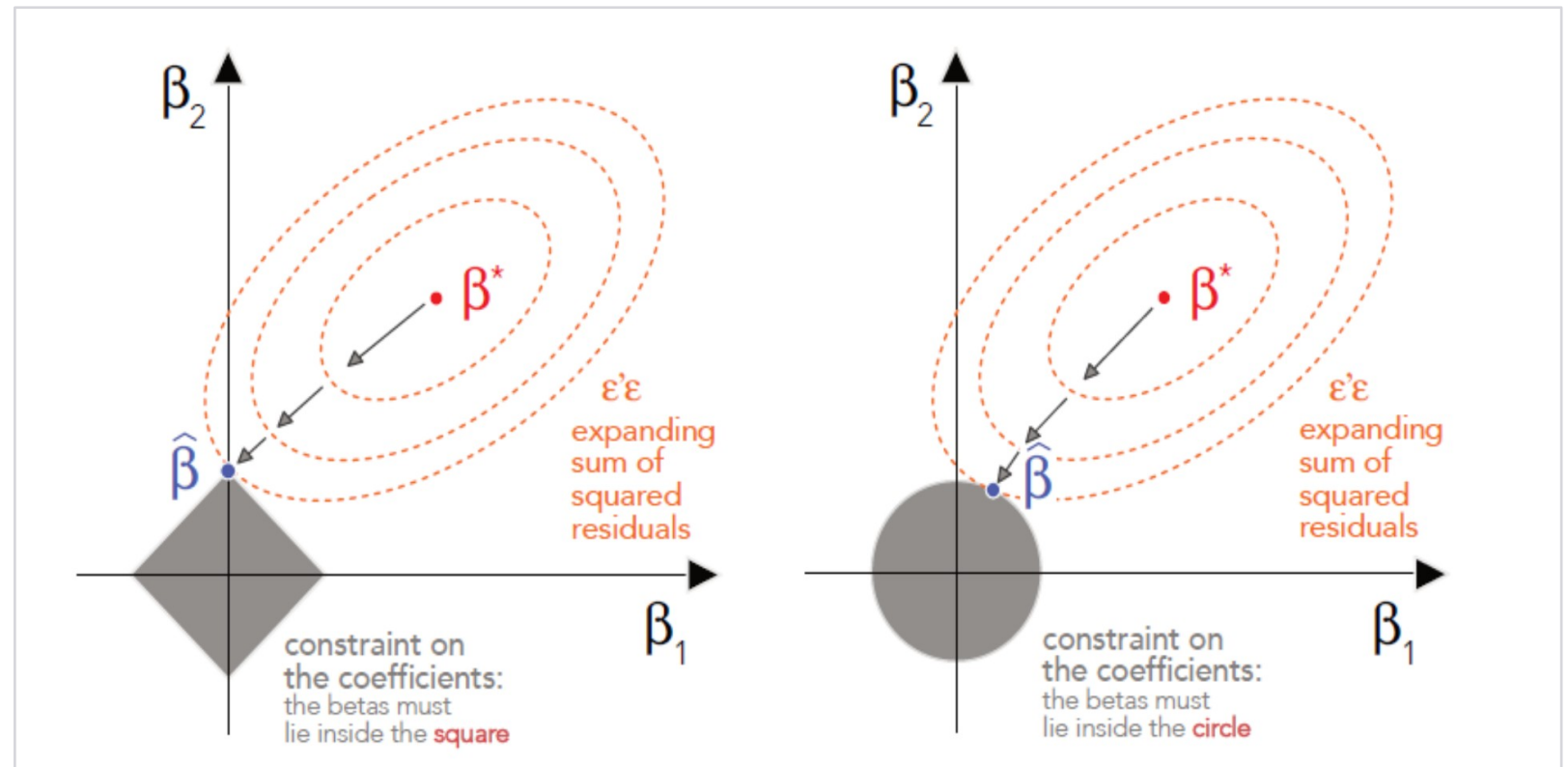
- No closed form: solved by coordinate descent
- Among correlated features it tends to keep one and drop the rest, somewhat arbitrarily
- Standardize first; λ = alpha in scikit-learn

Lasso vs. Ridge: The Geometry

Lasso when only a few features matter (easy to read); Ridge when many each contribute a little

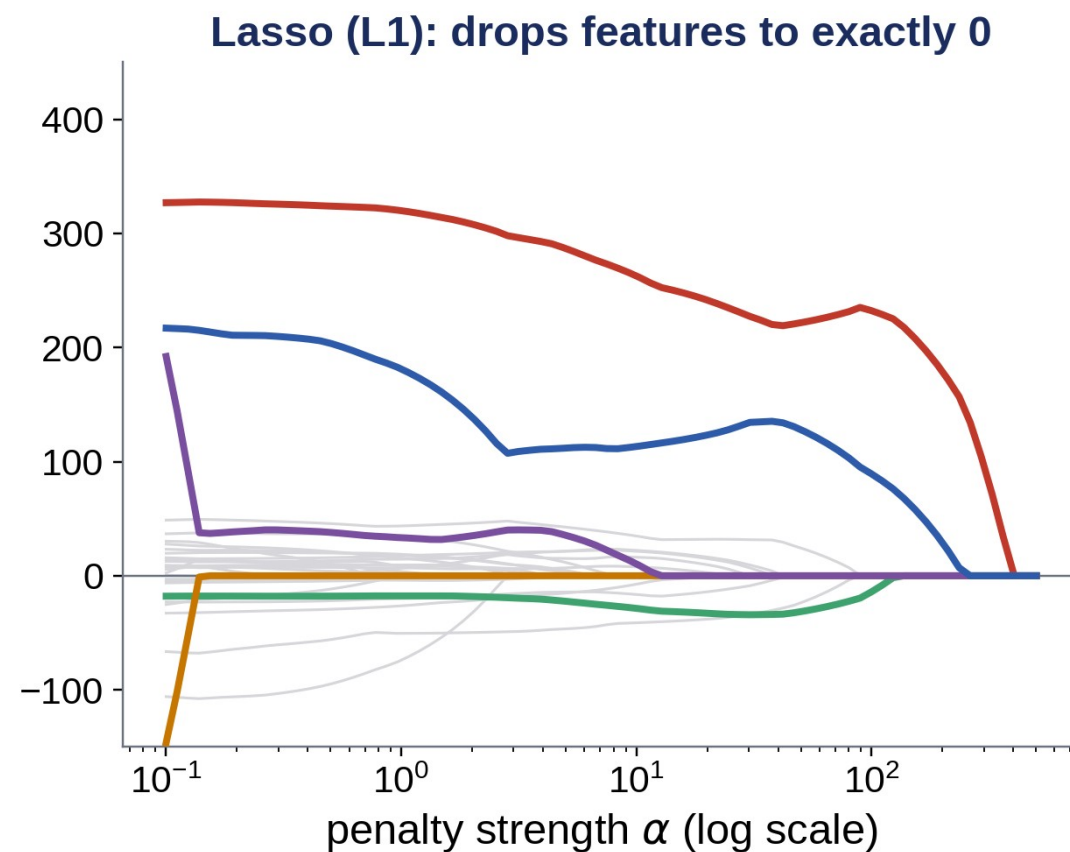
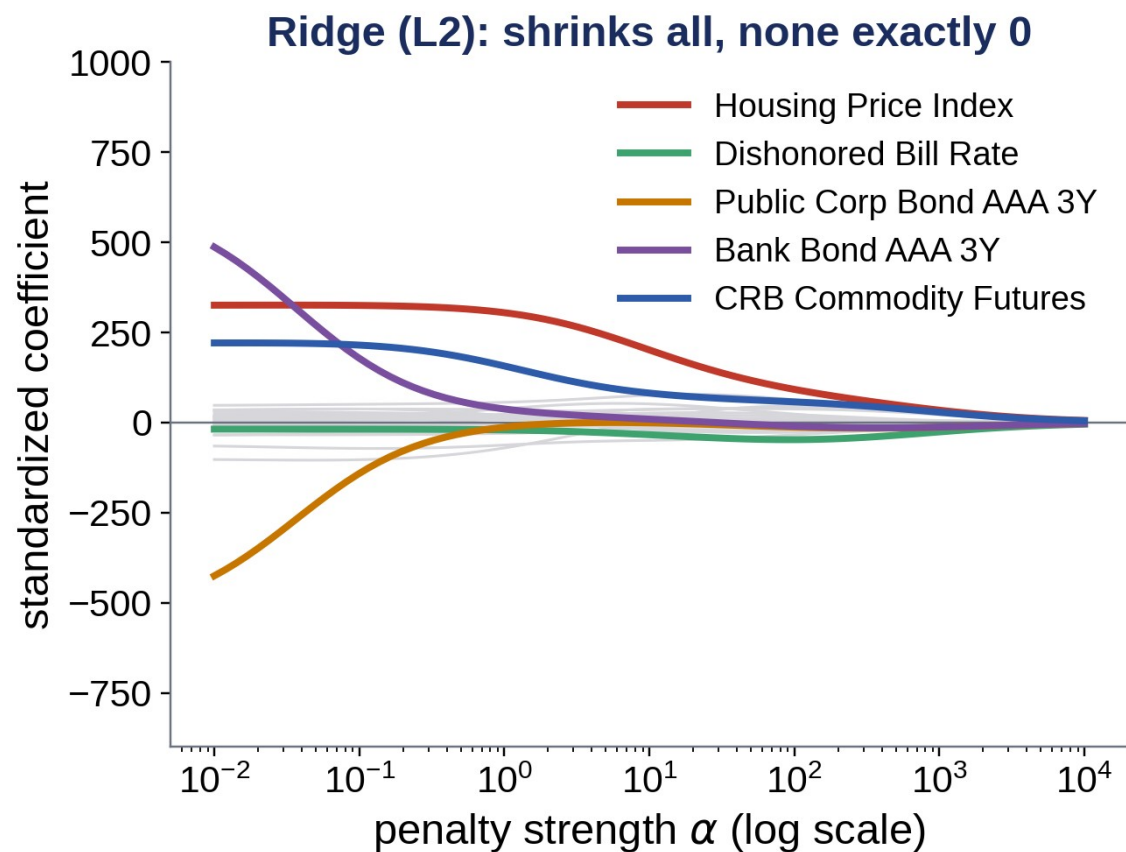
Read the chart

- Left: Lasso (diamond)
- Right: Ridge (circle)
- Ellipses: equal-error contours
- Diamond corners sit on an axis $\rightarrow$ a coefficient hits 0



Ridge vs. Lasso on the KOSPI Data

As α grows, Ridge shrinks every coefficient smoothly; Lasso switches features off one by one



ElasticNet: Ridge + Lasso

Combines both penalties — the same model family as glmnet in R (scikit-learn form shown)

$$\frac{1}{2m} \|\mathbf{y} - X\mathbf{w}\|_2^2 + \underbrace{\alpha \rho \|\mathbf{w}\|_1}_{\text{L1 part}} + \underbrace{\frac{\alpha(1 - \rho)}{2} \|\mathbf{w}\|_2^2}_{\text{L2 part}}$$

$$\alpha = \lambda_1 + \lambda_2 \quad \rho = \text{l1_ratio} = \frac{\lambda_1}{\lambda_1 + \lambda_2}$$

Two knobs

- alpha: overall strength
- l1_ratio = 1 → Lasso; 0 → Ridge

Naming trap: R vs. Python

- glmnet: alpha = mix, lambda = strength
- sklearn: l1_ratio = mix, alpha = strength

Regularized Models in scikit-learn

All three live in `sklearn.linear_model` (lower case — Python is case-sensitive)

Python · Colab

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.linear_model import ElasticNet

ridge = Ridge(alpha=1.0)           # L2
lasso = Lasso(alpha=1.0)           # L1
enet = ElasticNet(alpha=1.0, l1_ratio=0.5)

ridge.fit(X_train_std, y_train)    # scaled X
```

Fixed from the original

- `sklearn.Linear_model` → `sklearn.linear_model`

Same alpha ≠ same strength

$$\text{Ridge : } \| \mathbf{y} - X\mathbf{w} \|_2^2 + \alpha \| \mathbf{w} \|_2^2$$

$$\text{Lasso : } \frac{1}{2m} \| \mathbf{y} - X\mathbf{w} \|_2^2 + \alpha \| \mathbf{w} \|_1$$

So

- Lasso's loss is divided by $2m$; Ridge's is not
- Tune alpha by cross-validation, never by habit

Worked Example: Train, Predict, Compare

The original code, corrected — run on the standardized KOSPI features (random split, seed 42)

Python · Colab

```
from sklearn import linear_model
from sklearn.metrics import mean_squared_error
import math
ridge_model = linear_model.Ridge(alpha=1)
lasso_model = linear_model.Lasso(alpha=1)
ridge_model.fit(X_train, y_train)
lasso_model.fit(X_train, y_train)
ridge_pred = ridge_model.predict(X_test)
lasso_pred = lasso_model.predict(X_test)
ridge_rmse = math.sqrt(
    mean_squared_error(y_test, ridge_pred))
lasso_rmse = math.sqrt(
    mean_squared_error(y_test, lasso_pred))
```

Output

Ridge RMSE 107.04 • Lasso RMSE 103.36 (27 of 29 kept)

What was fixed

- `imort` → `import`
- `Ridge_modelfit` → `.fit`
- Duplicate Lasso line removed
- `sklearn_metrics` → `sklearn.metrics`
- Missing `')` added
- Boston data → KOSPI

05

SECTION 05

Polynomial Regression

Curves from a linear model — and the feature explosion that follows

Polynomial Regression: Still a Linear Model

'Linear' means linear in the coefficients w — the features themselves may be squared or multiplied

$$\hat{y} = w_0 + w_1x + w_2x^2 + \cdots + w_dx^d$$

$$(x_1, x_2, x_3) \xrightarrow{\text{degree 2}} (1, x_1, x_2, x_3, x_1^2, x_2^2, x_3^2, \underbrace{x_1x_2, x_1x_3, x_2x_3}_{\text{interaction terms}})$$

Interaction terms

- $x_1 \cdot x_2$ lets the effect of x_1 depend on x_2
- Often the most useful new terms in practice

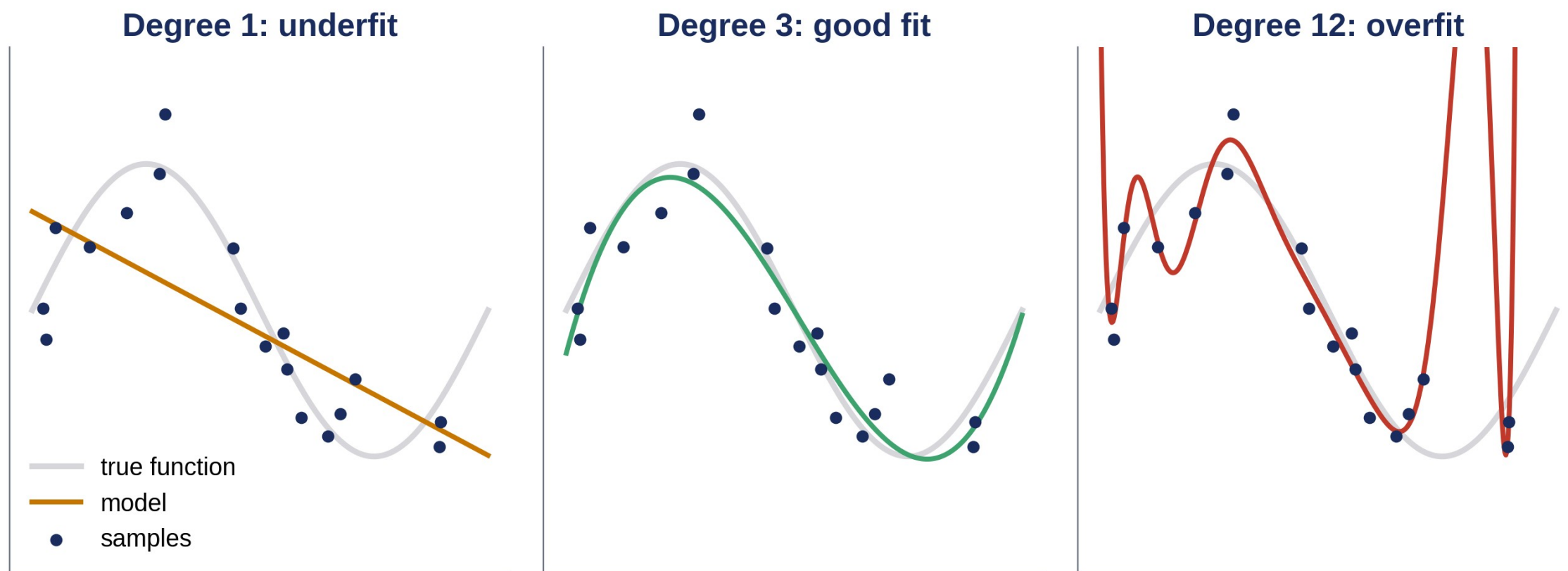
$$\# \text{ columns} = \binom{p+d}{d} = \binom{29+2}{2} = 465$$

465 columns vs. 128 training rows

- $p > n \rightarrow$ regularize; degree 2–3 is usually enough

Degree Controls Complexity

Degree 1 underfits, degree 3 captures the curve, degree 12 chases the noise between the points



PolynomialFeatures in scikit-learn

A transformer, not a model: fit the expansion on training data, reuse it on test data

Python · Colab

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2,
                          interaction_only=False,
                          include_bias=True)

poly_X_train = poly.fit_transform(X_train)
poly_X_test = poly.transform(X_test) # no fit!
print(poly_X_train.shape)           # (128, 465)
```

Then any linear model

```
lr = LinearRegression().fit(poly_X_train, y_train)
ridge = Ridge(alpha=1).fit(poly_X_train, y_train)
```

Parameters

- `interaction_only=True`: products only, no squares
- `include_bias` adds the column of 1s — set False with LinearRegression, which fits its own intercept
- Typo fixed: PolynomilaFeature → PolynomialFeatures

06

SECTION 06

KOSPI Case Lab

The Colab notebook, cell by cell: 160 months, 29 indicators, one index

The Data: 29 Indicators, 160 Months

Monthly, May 2003 – Aug 2016 · target = KOSPI month-end close · file: KOSPI_Index_EN.csv

Group (count)	Features
Sentiment (3)	FKI BSI, Construction BSI (outlook), OECD leading indicator (Korea)
Real economy & credit (5)	Housing price index, unemployment rate, dishonored bill rate, outbound travelers (level, YoY)
Trade (1)	Daily exports YoY
Rates & spreads (7)	MSB 364D, public-corp and bank AAA 3Y yields; two term spreads, two credit spreads
Market risk (1)	VKOSPI (KOSPI 200 volatility index)
National accounts (8)	Quarterly, repeated monthly: GDP services, investment, exports, demand, deflator, saving & investment rates
Commodities (4)	Reuters/CRB equal-weight: futures, energy, industrials, precious metals

Dates were not in the source file; they are inferred by matching KOSPI month-end closes (e.g., Oct 2008 = 1,113; Aug 2016 = 2,035).

Running the Notebook in Colab

KOSPI_Prediction_EN.ipynb — the next slides follow it cell by cell

Sections	What the cells do
§0 – §1	Setup, then load and inspect the data
§2	Random 80/20 split, then standardize on the training rows
§3 – §8	OLS, Ridge, Lasso and polynomial models, compared in one table

§0 Setup · Python · Colab (abridged)

```
import os
FILE = 'KOSPI_Index_EN.csv'
if not os.path.exists(FILE):      # first run in Colab
    from google.colab import files
    files.upload()                # pick the CSV

import math, numpy as np, pandas as pd
import matplotlib.pyplot as plt
```

In class

- Upload KOSPI_Index_EN.csv, then run the cells top to bottom
- Every number on the next slides is the notebook's own output
- KOSPI_Data_Dictionary.csv maps each column to its Korean source name

The Notebook Pipeline

Five steps from CSV to six fitted models

1 Load

read the CSV

```
pd.read_csv(...)
```

2 X / y

features vs.
target

```
X = df.drop(...)
```

3 Split

80 / 20

```
train_test_split
```

4 Scale

fit on train only

```
StandardScaler
```

5 Model

OLS, Ridge,
Lasso, poly

```
fit → predict
```

From the notebook (translated)

```
std = StandardScaler()  
X_train = std.fit_transform(X_train)  
X_test = std.transform(X_test)  # never fit here
```

Right instinct

- Test data must not leak into fitting

§1 Load and Inspect the Data

160 monthly rows · Date + 29 features + the target KOSPI

Python · Colab

```
kospi = pd.read_csv(FILE)
kospi.shape

X = kospi.drop(columns=['Date', 'KOSPI']) # features
y = kospi['KOSPI']                       # target
dates = pd.to_datetime(kospi['Date'])
print(X.shape, y.shape)
y.head(); y.tail()
```

Output

```
(160, 31)
(160, 29) (160,)
head:  633.4   669.9   713.5   759.5   697.5
tail: 1994.2  1983.4  1970.4  2016.2  2034.7
```

What to notice

- 31 columns = Date + 29 features + KOSPI
- KOSPI: 633 in May 2003, 2,035 in Aug 2016
- 8 national-accounts series are quarterly, repeated for each month of the quarter
- Seven groups: sentiment, real economy & credit, trade, rates & spreads, market risk, national accounts, commodities

§2 Split, Then Standardize on Training Data Only

Random 80/20 split as in the original (seed 42): 128 training rows, 32 test rows

Python · Colab

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
columns = X_train.columns

from sklearn.preprocessing import StandardScaler
std = StandardScaler()
X_train = std.fit_transform(X_train) # learn mean, sd
X_test = std.transform(X_test)      # reuse them
```

Resulting shapes

X_train (128, 29) X_test (32, 29)

Same random split as the original lab, so every number matches the notebook.

✓ fit on train, transform test

- Mean and standard deviation come from the 128 training rows
- The test set is scaled with those same numbers

✗ std.fit_transform(X_test)

- Test-set statistics leak into the model
- Scores look better than they will in real use

§3 Linear Regression in Four Steps

Instantiate → fit → predict → evaluate, then read the coefficients

Python · Colab

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error as mse
from sklearn.metrics import r2_score as r2

lr = LinearRegression()           # 1 instantiate
lr.fit(X_train, y_train)         # 2 fit
y_pred = lr.predict(X_test)      # 3 predict
print('RMSE:', math.sqrt(mse(y_test, y_pred)))
print('R^2 :', r2(y_test, y_pred) * 100) # 4

coef = pd.Series(lr.coef_, index=columns)
coef.sort_values(key=abs, ascending=False).head(6)
```

Output (rounded)

```
RMSE: 107.51      (6.8% of the average test level)
R^2 : 94.60
```

Feature (standardized)	coef_
Bank Bond AAA 3Y	+852.4
Public Corp Bond AAA 3Y	-728.9
Housing Price Index	+307.0
CRB Commodity Futures	+203.9
CRB Precious Metals	-90.3
Outbound Travelers	+70.7

Red flag

- Two near-identical yields (corr 0.997) get huge, offsetting weights → multicollinearity, slide 11

§4 – §5 Ridge and Lasso at $\alpha = 100$

Same four steps, one extra argument — Lasso switches 26 of the 29 features off

Ridge · Python · Colab

```
from sklearn.linear_model import Ridge
ridge_model = Ridge(alpha=100)
ridge_model.fit(X_train, y_train)
y_pred = ridge_model.predict(X_test)
print('RMSE:', math.sqrt(mse(y_test, y_pred)))
print('R^2:', r2(y_test, y_pred) * 100)
```

Output

```
RMSE: 159.27
R^2 : 88.16
all 29 coefficients non-zero
```

Lasso · Python · Colab

```
from sklearn.linear_model import Lasso
lasso_model = Lasso(alpha=100)
lasso_model.fit(X_train, y_train)
y_pred = lasso_model.predict(X_test)
print('RMSE:', math.sqrt(mse(y_test, y_pred)))
print('R^2:', r2(y_test, y_pred) * 100)
coef_lasso = pd.Series(lasso_model.coef_,
                        index=columns)
coef_lasso[coef_lasso != 0].round(1)
```

Output

```
RMSE: 173.49      R^2 : 85.95
Housing Price Index      222.4
Dishonored Bill Rate     -22.7
CRB Commodity Futures     85.8
```

Both trail OLS on this split (RMSE 107.5): $\alpha = 100$ was set by habit, not tuned by cross-validation.

§6 – §7 Polynomial Features, Then Regularize

Degree 2 turns 29 columns into 465 — more columns than the 128 training rows

Python · Colab (three model cells condensed)

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
poly_X_train = poly.fit_transform(X_train)
poly_X_test = poly.transform(X_test) # no refit
print(poly_X_train.shape, poly_X_test.shape)

for model in [LinearRegression(),
               Lasso(alpha=100), Ridge(alpha=1)]:
    model.fit(poly_X_train, y_train)
    y_pred = model.predict(poly_X_test)
```

Output

(128, 465) (32, 465)

Model	RMSE	Test R ²	Non-0
Poly-2 + OLS	129.1	0.922	464
Poly-2 + Lasso $\alpha=100$	169.7	0.866	7
Poly-2 + Ridge $\alpha=1$	124.7	0.927	464

Read it

- Train $R^2 = 1.000$ for OLS and Ridge: the 128 rows are memorized
- Lasso keeps 7 of 465 terms and still scores 0.87
- Ridge $\alpha = 1$ edges out OLS on test RMSE (124.7 vs. 129.1)

§8 One Helper, Six Models

`evaluate()` refits every model on the same split and reports train R^2 next to test R^2

Python · Colab

```
def evaluate(models, Xtr, Xte, ytr, yte, Ptr=None, Pte=None):
    rows = []
    for name, model, use_poly in models:
        a, b = (Ptr, Pte) if use_poly else (Xtr, Xte)
        model.fit(a, ytr)
        yp = model.predict(b)
        rows.append({'model': name,
                    'test RMSE': math.sqrt(mse(yte, yp)),
                    'test R2': r2(yte, yp),
                    'train R2': r2(ytr, model.predict(a)),
                    'non-zero coefs':
                        int(np.sum(np.abs(model.coef_) > 1e-9))})
    return pd.DataFrame(rows).set_index('model').round(3)

MODELS = lambda: [
    ('Linear regression', LinearRegression(), False),
    ('Ridge a=100', Ridge(alpha=100), False),
    ...
    ('Poly-2 + Ridge a=1', Ridge(alpha=1), True)]
res_random = evaluate(MODELS(), X_train, X_test, y_train,
                     y_test, poly_X_train, poly_X_test)
```

Why a helper?

- Same split and metrics for every model
- Train R^2 beside test R^2 exposes overfitting
- MODELS is a lambda: each call hands out fresh, unfitted models
- Pass a different split and it runs unchanged
- Results → next slide

Results as in the Notebook (Random Split)

All six models on the same random split (seed 42) — test R^2 next to train R^2

Model	Test RMSE	Test R^2	Train R^2	Non-zero w
Linear regression	107.5	0.946	0.968	29
Ridge $\alpha = 100$	159.3	0.882	0.909	29
Lasso $\alpha = 100$	173.5	0.860	0.835	3
Poly-2 + OLS	129.1	0.922	1.000	464
Poly-2 + Lasso $\alpha = 100$	169.7	0.866	0.850	7
Poly-2 + Ridge $\alpha = 1$	124.7	0.927	1.000	464

Lasso $\alpha = 100$ keeps only three features: Housing Price Index (+222), CRB Commodity Futures (+86), Dishonored Bill Rate (−23)

Summary

1

OLS minimizes squared error

Normal equation or gradient descent; watch for multicollinearity ($X'X$ near-singular).

2

Evaluate out of sample, honestly

RMSE / MAE / R^2 on a held-out test set the model never saw.

3

Regularize to match complexity to data

Ridge shrinks, Lasso selects, ElasticNet does both — tune α by cross-validation.

Take-home: run the Colab notebook and reproduce every number on these slides.